Epistemic Capture and the Action Boundary: Corrigibility for Learned and Agentic Systems

Anivar A Aravind
Independent Researcher
Bengaluru, India
ping@anivar.net

February 2026 (Working Draft)

Abstract

This paper extends the corrigibility framework to learned and agentic systems, where deterministic rule execution is replaced by probabilistic inference. Three structural pressures constrain corrigibility in learned systems: opacity of inference (requiring LWD-R transparency), concentration of training resources (compute capture), and acceleration of automated action (governance denial of service). The action boundary protocol separates probabilistic inference from deterministic execution. The invariant holds: systems remain corrigible if and only if the five conditions close the loop under stochastic verification.

Dependency. This paper presupposes the companion paper [Companion Paper, 2026], which derives the five corrigibility tests (EXIT, CODE, AUDIT, GOVERN, FORK) from control theory and commons governance. The present paper does not re-derive these tests; it extends their verification methods to stochastic systems.

Keywords: Epistemic Public Infrastructure, AI Governance, Corrigibility, Action Boundary Protocol, Compute Capture, LWD-R

How to Read This Paper. Corrigibility is the minimal architectural condition that keeps public systems reversible. The argument has three parts: learned systems require bounded error propagation (the conceptual invariant); five control-layer conditions extend to stochastic inference (the architectural constraints); and these constraints discriminate between AI-mediated systems (the applied evaluation). Case studies illustrate discriminative power, not exhaustive institutional evaluation.

Executive Summary

When infrastructure incorporates machine learning, it no longer merely processes transactions. It generates claims about reality. This creates **epistemic public infrastructure (EPI)**.

Traditional transparency is insufficient. Reconstruction of model behavior requires disclosure of logic, weights, data provenance, and representational schema (**LWD-R**). Practical forkability is constrained by compute asymmetry. Performance degrades under distributional shift.

The **action boundary protocol** separates probabilistic inference from deterministic execution through enforceable validation layers. Without such boundaries, agentic systems exercise unbounded epistemic power.

1 Introduction

The companion paper [Companion Paper, 2026] establishes a corrigibility framework for Digital Public Infrastructure (DPI), deriving five structural tests from cybernetics, commons governance, and free software. That framework addresses **deterministic** infrastructure: ledgers, registries, and payment rails where source code is the arbiter of behavior. This paper extends the framework to **stochastic** infrastructure, what we term **Epistemic Public Infrastructure (EPI)**: systems where state decisions emerge from learned parameters rather than explicit logic.

Definition 1.1 (Epistemic Public Infrastructure)**.** EPI comprises AI systems that determine citizen outcomes at population scale, including: identity verification models, welfare eligibility systems, public-facing chatbots with administrative authority, and autonomous agents operating within mandatory infrastructure.

1.1 DPI and EPI: Structural Contrast

Digital Public Infrastructure (DPI) processes transactions through deterministic logic. Epistemic Public Infrastructure (EPI) produces classifications, predictions, or inferences through learned parameters.

Table 1: Structural contrast between DPI and EPI

DPI	**EPI**
Deterministic rule execution	Stochastic learned inference
Logic transparency sufficient	Weights, data, and representations matter
Forkability tied to software	Forkability tied to compute capacity
Static certification possible	Performance degrades under distribution shift

Corrigibility in EPI therefore requires additional structural conditions beyond those sufficient for deterministic systems.

Why "Epistemic"? The term is structurally precise. In deterministic DPI, infrastructure manages *what happened*: the **Ledger State** (a payment was made, an identity was verified). In learned systems, infrastructure manages *what is true*: the **Inference State** (a person is "eligible," a transaction is "fraudulent," a citizen is a "risk"). This shift from transactional facts to administrative knowledge constitutes a transfer of **epistemic sovereignty**.

When a state uses a model to determine eligibility, it is no longer merely processing data; it is producing a *claim to knowledge*. If that claim is hidden in a latent space, the infrastructure has captured the state's ability to know or contest the truth. The "R" in the LWD-R Triad (Representation, Section 3) is the epistemic anchor: if the categories through which the state "sees" the world are proprietary and non-contestable, the state has delegated its interpretive sovereignty. Citizens are not merely locked out of a database; they are locked into a reality they cannot define.

By naming this domain **Epistemic Public Infrastructure**, the framework asserts that the "truth" produced by an AI system must satisfy the same corrigibility tests as a legal fact. The question shifts from "Does the AI work?" to "Is the knowledge produced by this system structurally falsifiable and reversible?"

Architectural Closure. DPI solves the problem of **transactions** (Scale and Velocity). EPI solves the problem of **meaning** (Stochasticity and Representation). Once the infrastructure for creating administrative reality (EPI) is bounded by the same five structural tests as the

infrastructure for processing payments (DPI), the framework achieves closure. The theory has scaled from the registry to the world model.

The five structural tests are:

- **EXIT**: Reversibility of participation
- **CODE**: Inspectability of logic
- **AUDIT**: Independent verification
- **GOVERN**: Binding constraint
- **FORK**: Capacity to reproduce

The Cybernetic Foundation. These tests are not arbitrary normative preferences but derive from control theory. Ashby's Law of Requisite Variety [Ashby, 1956] establishes that a controller must possess at least as many response states as the system it governs. Applied to infrastructure: governance mechanisms must match the variety of harms a system can produce. The five tests operationalize this requirement as a closed feedback loop: EXIT provides the error signal (affected parties can refuse), AUDIT acts as the sensor (errors are detectable), CODE ensures transmission clarity (logic is inspectable), GOVERN serves as the actuator (correction is binding), and FORK provides selection pressure over the entire loop (the system can be reproduced if operators refuse correction).

The Open-Loop Instability Theorem. When any test fails, the feedback loop opens. Open-loop systems inevitably drift toward instability because errors accumulate without correction. The companion paper proves formally that partial compliance (meeting some tests while failing others) is functionally equivalent to complete failure. A system with EXIT but no GOVERN allows users to detect harm but not correct it; a system with CODE but no FORK allows inspection but not competition. The five tests form an *irreducible* set: removing any one collapses the corrective apparatus.

This paper stress-tests the framework against learned systems: infrastructure where behavior is determined by parameters learned from data (AI/ML) rather than explicit code logic. If the five tests are truly fundamental, they must apply not only to deterministic infrastructure but also to stochastic systems where traditional verification assumptions collapse.

The Challenge. Learned systems dismantle the assumption that source code is the sole arbiter of behavior. In AI, identical inputs may yield stochastic outputs; behavior is defined by training data; and opacity is often inherent rather than contrived. Yet the collapse of deterministic transparency does not invalidate corrigibility's structural necessity. As systems become less intelligible, external correction becomes *more* critical.

Thesis. The five corrigibility tests remain invariant for learned systems. What changes is the verification method and the resource barriers to compliance.

Scope. This paper addresses Epistemic Public Infrastructure (Definition 1.1): AI systems deployed at population scale with administrative authority. It does not address consumer AI products where exit is trivial, nor research systems without deployment impact.

Paper Structure. Section 2 establishes that the five tests remain invariant. Section 3 extends the CODE test via LWD-R disclosure. Section 4 introduces Variety Drift. Section 5 analyzes Compute Capture. Section 6 examines agentic scaling and GDoS. Section 7 addresses workflow sovereignty and WCC. Section 8 provides architecture-specific risk analysis. Section 11 discusses implications and limitations.

Key Contributions.

1. **Epistemic Public Infrastructure (EPI)**: Distinguishes AI-mediated administration from deterministic DPI
2. **LWD-R**: Four-layer transparency (Logic, Weights, Data, Representation)
3. **Ontological Capture**: When a model's categories become non-contestable facts
4. **Variety Drift**: Why AI corrigibility is a persistence condition
5. **Compute Capture**: When "open weights" fails FORK
6. **GDoS**: Governance collapse under agentic scaling
7. **WCC**: Quantifies epistemic delegation risk
8. **Action Boundary**: Deterministic envelope around stochastic inference

2 The Invariance of the Standard

Proposition 2.1. The five corrigibility tests remain invariant for learned systems. What changes is the verification method.

The structural requirement for corrigibility derives from Ashby's Law of Requisite Variety: a controller must have at least as many response states as the system it governs [Ashby, 1956]. This requirement is independent of whether the controlled system is deterministic or stochastic. An AI model that affects citizen outcomes at population scale generates the same variety of potential harms as deterministic infrastructure; therefore, the governance apparatus must possess equivalent corrective variety.

The table below illustrates how the mechanical verification of each test adapts to the probabilistic nature of AI without softening the structural requirement.

Test	DPI Verification	EPI Verification	Key Addition
EXIT	Can refuse the system	Disclosure + non-AI alternative; appeal to human guaranteed	Human fallback
CODE	Source code determines behavior	LWD-R: Logic, Weights, Data, Representation layers disclosed	**R** (ontology)
AUDIT	Inspect logic and outputs	Statistical bounds + continuous monitoring; variety drift measurement	Drift tracking
GOVERN	Maintainer and RFC process	Governance over objectives, value tradeoffs, and category definitions	Ontology governance
FORK	Copy code and deploy	Resource forkability: compute + data + training pipeline	Compute access

Table 2: Verification methods for DPI (deterministic) vs. EPI (learned) systems. The standard remains invariant; verification adapts.

Structural Pressures in Learned Systems. Three additional pressures constrain corrigibility in learned systems. Table 3 maps each pressure to its architectural response.

3 Transparency Failure in Learned Systems

In deterministic infrastructure, disclosure of operative logic is sufficient to enable inspection. In learned systems, logic is distributed across architecture, parameters, training data, and rep-

Pressure	What Breaks	What Fixes	Layer
Epistemic Opacity	CODE (inspection fails)	LWD-R transparency	Observability
Compute Capture	FORK (reproduction gated)	Training forkability	Replacement
Acceleration Overwhelm	GOVERN (decisions outpace review)	Action Boundary	Constraint

Table 3: Three structural pressures in learned systems: what corrigibility condition they break, what architectural mechanism restores it, and which control layer is affected.

resentational schemas. Disclosure of source code alone does not reveal the operative decision boundary.

Principle 3.1 (Source Distribution)**.** Publishing inference code alone does not satisfy the CODE test. It allows an auditor to run the machine, but not to understand why it produces specific outputs.

To satisfy the CODE condition in learned systems, four components must be disclosed:

- **Logic (L)**: Model architecture and inference procedures
- **Weights (W)**: Learned parameters
- **Data (D)**: Training datasets with provenance
- **Representation (R)**: Ontological categories and label schemas

We refer to this as **LWD-R transparency**. Omission of any component prevents meaningful external reconstruction of epistemic behavior.

To bridge this gap, compliance requires structured disclosure across the training pipeline, such as **Data Sheets for Datasets** [Gebru et al., 2021] and **Model Cards** [Mitchell et al., 2019]. These artifacts function as the "source code" for structural evaluation; withholding them renders the system a black box.

3.1 The LWD-R Disclosure Requirement

For world models and agentic systems, the traditional transparency requirements (code, weights, data) are necessary but insufficient. Consider what it means to "inspect" a welfare eligibility model: seeing the inference code tells you how predictions are computed, but not *why* certain applicants are classified as high-risk. That "why" is encoded across four distinct layers.

Definition 3.1 (LWD-R: The Four Layers of AI Transparency)**.** A learned system satisfies LWD-R disclosure if the following artifacts are publicly available:

- **L (Logic)**: Model architecture and inference code (*how* predictions are computed)
- **W (Weights)**: Trained parameters (the learned patterns that produce outputs)
- **D (Data)**: Training corpus with provenance (*what* the model learned from)
- **R (Representation)**: The categorical schema (*how the model sees the world*)

Contestability Criterion. Transparency without contestability is surveillance. LWD-R disclosure is necessary but not sufficient: affected parties must have structural mechanisms to challenge representational categories, not merely observe them. A system publishing full LWD-R while providing no pathway for category contestation satisfies observability but fails constraint-layer corrigibility.

Why Four Layers? The first three (L, W, D) are familiar from "open source AI" discourse. The fourth, **Representation (R)**, is this framework's principal contribution to AI transparency discourse.

Why "R" Is the Novel Contribution. Existing "open AI" frameworks focus on L, W, and D. The Representation layer addresses a distinct failure mode: a model may publish weights, code, and data manifests while encoding *non-contestable categories* in its latent space. When an eligibility model classifies citizens as "high-risk," that category becomes an administrative fact. If the category itself cannot be challenged, the model has captured the state's interpretive sovereignty.

R is not transparency about *how* the model works; it is transparency about *how the model sees the world*. These categories (the model's ontology) are often hidden in the latent space, undocumented and non-contestable. A specific form of opacity arises when representational categories themselves become uncontestable ("ontological capture"), weakening constraint-layer contestability. When a model's classifications cannot be challenged by affected parties, the state has surrendered interpretive sovereignty regardless of weight publication.

3.2 Machine-Verifiable Legitimacy

For learned and agentic systems, corrigibility must be paired with machine-verifiable legitimacy: (a) *authority encoding*—delegation and scope expressed as machine-interpretable credentials; (b) *revocation propagation*—immediate, observable revocation states that downstream execution environments respect; (c) *evidence standards*—tamper-evident execution traces and evaluation artifacts that meet defined admissibility criteria; and (d) *incident-triggered mitigation*—automated mitigation gating and mandatory rollback protocols when thresholds breach. These are operational requirements; without them, architectural constraints remain unproven. Existing governance frameworks such as [Abadie, 2026] provide workflow orchestration but lack these machine-verifiable legitimacy requirements.

4 Variety Drift: Why AI Corrigibility Is a Persistence Condition

Traditional software remains static until explicitly patched. A tax calculation program written in 2020 will compute the same result in 2030, unless someone changes the code. Learned systems are different: they are *frozen snapshots* of a world that keeps moving.

The Problem. An AI model trained on 2024 data encodes 2024 patterns. By 2026, the world has changed: new fraud techniques emerge, economic conditions shift, language evolves. The model's "variety" (its capacity to handle diverse situations) was fixed at training. The world's variety keeps expanding. The gap between them grows over time.

Definition 4.1 (Variety Drift)**.** Let $V_{\text{world}}(t)$ be the variety of situations the environment presents at time t, and $V_{\text{model}}(\theta)$ be the variety the model can handle, fixed at training. **Variety Drift** is the growing gap:

$$\Delta(t) = V_{\text{world}}(t) - V_{\text{model}}(\theta) \tag{1}$$

Why This Matters. Without mechanisms for retraining accessible to the community, variety drift widens indefinitely. A model that passed all five corrigibility tests at deployment may fail them two years later. Not because anyone changed anything, but because the world moved and the model did not. **AI corrigibility is not a deployment certification; it is a persistence condition.**

The Revalidation Threshold. When variety drift exceeds a domain-specific threshold τ, mandatory revalidation is triggered:

$$\Delta(t) > \tau \implies \text{Revalidation Required} \tag{2}$$

The threshold τ is not universal; it depends on consequence severity. High-stakes domains (welfare eligibility, criminal risk assessment) require lower thresholds than low-stakes domains (content recommendation). Setting τ is a governance decision, not a technical one.

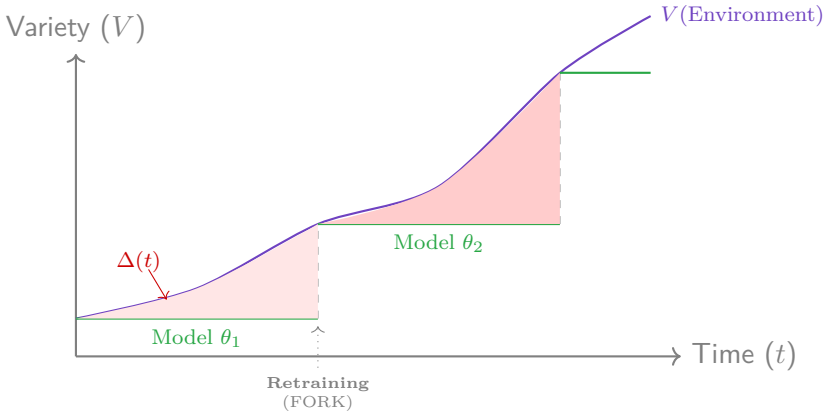

Figure 1: **The Temporal Variety Gap.** Environmental variety $V(e)$ evolves continuously (curve). A fixed model θ (stepped line) diverges from reality. The shaded area represents accumulated error. Only **FORK** rights allow the gap to be closed via retraining.

Claim 4.1. A learned system that passes all five corrigibility tests at deployment may fail them over time if the community lacks the resources to retrain. Corrigibility for AI is a dynamic property, not a static certification.

5 Resource Barriers to Forkability

In traditional software, forking is cheap: copy the code, set up a server, deploy. The cost is measured in storage and labor. In AI, forking is expensive: the cost is measured in *compute*, the GPU-hours required to retrain a model from scratch.

The Structural Difference. When Elastic changed its license, AWS forked Elasticsearch and the community continued development. This was possible because the resource barrier was low. When Meta releases Llama weights, can the community "fork" Llama in the same sense? They can *run* it (inference). They can *adapt* it (fine-tuning). But can they *reproduce* it from scratch if Meta's training data turns out to be problematic? For frontier models, the answer is no: the compute cost exceeds what any non-corporate actor can access.

Layer	**Cost**	**Barrier**
Inference code	Minimal	None
Running open weights	Low	Minor
Fine-tuning	Moderate	Economic
Retraining from scratch	Very high	Structural

Table 4: Resource barriers to forking learned systems

Claim 5.1. Legal permission to fork without access to compute or data is meaningless. This is an infrastructure question, not a licensing question.

5.1 Compute Capture

The Problem. A company releases model weights under an open license. Headlines announce "open source AI." But the training run cost $100 million in compute. No university, no government research lab, no coalition of independent researchers can afford to retrain it. If the model exhibits problematic behavior traceable to training data, the community can observe the symptoms but cannot fix the cause. They can run the model; they cannot reproduce it.

Definition 5.1 (Compute Capture)**. Compute Capture** occurs when the compute cost to retrain a model (C_{train}) exceeds the compute accessible to non-operator actors ($C_{\text{accessible}}$) by more than a policy-determined threshold:

$$C_{\text{train}} > \kappa \cdot C_{\text{accessible}} \tag{3}$$

where κ is an accessibility multiplier (typically $1 \leq \kappa \leq 2$). Under Compute Capture, releasing weights provides **inference forkability** (you can run it) without **training forkability** (you can reproduce it).

Why This Matters. Legal permission to fork is meaningless without physical capacity to fork. If training cost exceeds 100× global academic compute capacity, the FORK test fails regardless of licensing terms. This is an infrastructure question, not a licensing question.

The formal FORK determination for training becomes:

$$\text{FORK}_{\text{training}} = \mathbf{1}[C_{\text{train}} \leq \kappa \cdot C_{\text{accessible}}] \wedge \mathbf{1}[\text{data\_manifest\_available}] \tag{4}$$

where $\mathbf{1}[\cdot]$ is the indicator function.

5.2 Compute as Governance Infrastructure

This analysis implies that **compute is itself DPI**: a common-pool resource requiring multi-stakeholder governance rather than unilateral corporate control. Legal forkability is insufficient absent practical compute accessibility. Architectural responses include:

1. **Public Compute Nodes**: State-sponsored clusters with multi-stakeholder governance
2. **Mandatory Cost Disclosure**: FLOP estimates and data manifests
3. **Federated Training Pools**: Distributed training across independent nodes

5.3 Minimum Evidence for Learned System Forkability

To prevent "FORK fails" from becoming a rhetorical claim rather than a verifiable determination, learned systems must satisfy specific evidentiary requirements:

Forkability Determination. A learned system passes FORK if:

1. All "Required" artifacts are publicly available under permissive terms
2. The compute cost estimate demonstrates feasibility for at least one non-operator entity
3. Either a successful third-party reproduction exists, OR the training code + data manifest enables reproduction in principle

5.4 Inference vs. Training Forkability

Systems that publish weights but withhold training data or code achieve only **inference forkability**, not **training forkability**. Inference forkability permits running the model; training forkability permits correcting it. Only the latter satisfies FORK for governance purposes.

Table 5: Evidence requirements for FORK in learned systems

Artifact	Status	Verification
Inference code	Required	Open source license; build reproducibility
Model weights	Required	Public download; hash verification
Training data manifest	Required	Data Sheets [Gebru et al., 2021] with provenance
Training checkpoint	Recommended	Enables fine-tuning without full retrain
Training code	Required	Includes preprocessing, hyperparameters
Compute cost estimate	Required	FLOP estimate or cloud cost documentation
Successful reproduction	Decisive	Third-party retrain within 10% of benchmark

Concrete Examples. As of 2026, most "open" models achieve only inference forkability:

- **Inference forkability only:** Llama 3 (Meta), DeepSeek, Gemma (Google), Mistral. Weights are published, but training data remains proprietary and full training code is not disclosed. Users can run and fine-tune these models but cannot independently reproduce or audit the training process.
- **Training forkability:** OLMo (Allen Institute), Pythia (EleutherAI). These projects publish weights, complete training code, and full data manifests (Dolma and The Pile respectively). Third parties have successfully reproduced training runs, satisfying the FORK test.

The distinction matters: when Llama or Gemma exhibit unexpected behavior, the community can observe symptoms but cannot diagnose root causes in training data. When OLMo exhibits unexpected behavior, researchers can trace the issue to specific training examples. **Open weights with closed training pipelines is inference transparency without correction capacity.**

Alignment with OSI Definition. The Open Source Initiative's Open Source AI Definition 1.0 [OSI, 2024] codifies this distinction, requiring: (1) data information sufficient for recreation, (2) complete training code, and (3) model parameters. By this standard, Llama, Gemma, DeepSeek, and Mistral do not qualify as open source AI despite marketing claims. The EU AI Act [EU AI Act, 2024] similarly distinguishes between models with varying transparency obligations. This framework's FORK test operationalizes the same principle: without training forkability, governance correction is structurally impossible.

6 Agentic Scaling and Governance Denial of Service

The preceding sections establish that learned systems face unique barriers to corrigibility: opaque training pipelines (violating CODE), frozen variety (the Temporal Variety Gap), and compute-gated reproduction (violating FORK). These challenges intensify as AI moves from isolated models to autonomous agents operating within infrastructure. As DPI transitions to support the "Agentic Economy," corrigibility moves from a moral preference to an engineering requirement. Autonomous agents lack the biological resilience to handle bureaucracy.

Principle 6.1 (Structural vs. Alignment Distinction)**.** This framework distinguishes structural corrigibility from "AI Alignment." The AI safety literature has extensively studied alignment (ensuring systems pursue operator-intended goals) [Soares & Fallenstein, 2017] and the "off-switch"

problem [Hadfield-Menell et al., 2017]. These framings center on *operator control*. Alignment asks: *Can the operator control the system?* Corrigibility asks: *Can the affected subject correct the system?* Structural legitimacy requires the latter.

6.1 Corrigibility as an API Contract for Agents

- **EXIT is Exception Handling**: To an agent, a mandatory system with no exit path appears as a logic deadlock.
- **CODE is Documentation**: Agents require inspectability of logic to form valid plans.
- **AUDIT is Observability**: An optimization agent cannot function without a reliable failure signal.
- **GOVERN is Constraint Discovery**: Advisory guidelines result in undefined behavior for software agents.

Claim 6.1. Incorrigible infrastructure is structurally incompatible with AI automation.

6.2 The Human Friction Buffer

Incorrigible infrastructures may appear stable under human load because humans absorb friction through delay, compliance, and informal workaround. Autonomous agents remove this buffering layer.

Illustrative Scenario. Consider an autonomous procurement agent that interacts with a mandatory payment API. On encountering a compliance flag, the agent receives an opaque error code. No machine-readable error taxonomy exists, and dispute resolution is human-mediated. The agent retries, triggering repeated failures and queue saturation. Human agents absorb such friction via manual override and phone calls; fleets of autonomous agents cannot. The result is a governance denial-of-service (GDoS) that cascades across the payment rail.

Governance Denial of Service. Under agentic scaling, governance capacity can be temporarily saturated ("governance denial of service"). When autonomous agents encounter errors, they retry programmatically rather than waiting patiently. Governance mechanisms designed for human patience collapse under machine speed. This is a constraint-layer failure that the Action Boundary must mitigate.

6.3 Agentic Interoperability Requirements

For an autonomous agent $\mathcal{A}$ to operate reliably within infrastructure $\mathcal{I}$, the following predicates must be strictly True:

1. **Reversibility:** $\forall$ state $s_t \in \mathcal{S}$, $\exists$ transition s_{t+1} (via `EXIT`) such that $\text{Cost}(s_{t+1}) < \infty$. (No deadlocks).
2. **Decidability:** $\forall$ input x, $\text{Compute}(x) \to \{0, 1\}$ is inspectable via **CODE**. (No hidden constraints).

Failure of these predicates renders $\mathcal{I}$ a "Hazmat Environment" for automated agents, effectively blocking the agentic economy.

6.4 The Action Boundary Protocol

As infrastructure transitions to the Rule of the Workflow, the fundamental unit of governance can no longer be the static database record, nor can it be the stochastic model weight. To subject autonomous systems to democratic constraint, the architecture must adopt a new structural primitive: the **Action Boundary**.

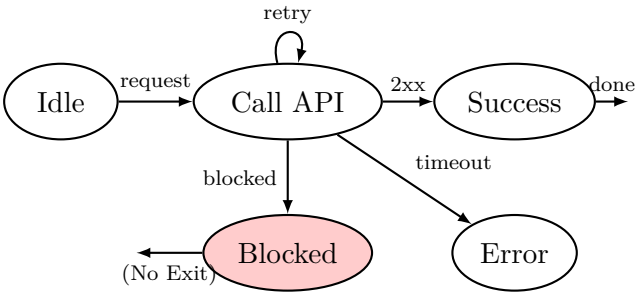

Figure 2: **Agentic automaton.** Without EXIT and observable error codes, agent loops indefinitely, times out, or escalates incorrectly.

Principle 6.2 (Action Boundary Protocol)**.** If the state cannot govern the neural network's weights, it must govern the **action boundary**: the deterministic envelope that wraps stochastic inference. The Action Boundary Protocol specifies: (1) hard-coded preconditions, (2) deterministic tool execution, and (3) explicit termination criteria to prevent Governance Denial of Service (GDoS).

In agentic systems, learned inference should not directly trigger irreversible action. Instead, inference outputs must pass through a deterministic validation layer that enforces policy constraints and safety rules.

Architectural Proposals and the Legitimacy Gap. Recent architectural proposals such as the DPI-AI Framework [Abadie, 2026] advance composable AI capabilities ("AI Blocks") orchestrated through "DPI Workflows." This framing correctly treats AI as modular infrastructure rather than institutional magic. However, composability of *capability* must be matched by composability of *legitimacy*. If AI Blocks are callable, authority must also be callable, bounded, signed, and revocable. If workflows orchestrate identity, policy, and inference, then risk must be tiered explicitly: informational assistance is not equivalent to a benefits denial; conditional automation is not equivalent to a rights-affecting determination. The Action Boundary Protocol provides this structural requirement: probabilistic inference separated from deterministic execution through enforceable validation layers.

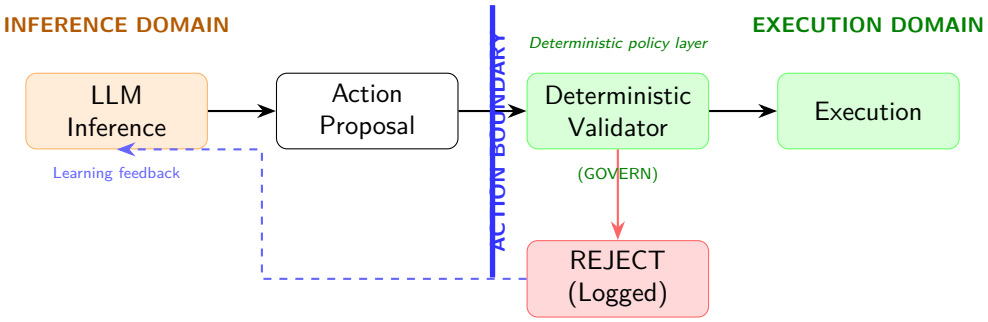

Figure 3: **Action Boundary Protocol.** Probabilistic inference (orange) produces action proposals. The **deterministic policy layer** (green, implementing GOVERN) enforces hard constraints before execution. Invalid actions are rejected, logged, and fed back for learning. The vertical **Action Boundary** separates the inference domain from the execution domain: the architectural instantiation of corrigibility in agentic systems.

The validator enforces hard constraints independent of probabilistic inference. Corrigibility in agentic systems therefore requires separation between epistemic generation and action authorization.

The engineering instantiation of this protocol already exists: the **Agentic Skills** standard [Agent Skills, 2025].

Agentic Skills (agentskills.io). The open `SKILL.md` specification defines a bounded, machine-readable orchestration protocol that encapsulates stochastic inference within deterministic action boundaries. Adopted by 18+ AI agent platforms (Claude, OpenAI Codex, GitHub Copilot, Cursor, and others), this standard provides exactly the governance primitive required for the Action Boundary Protocol. We adopt it here as the technical instantiation of our theoretical framework.

The SBOM Analogy. For security-minded readers: Agentic Skills function like a **Software Bill of Materials (SBOM)** for AI actions. Just as SBOMs enumerate software dependencies to enable vulnerability auditing, skill manifests enumerate action boundaries to enable governance auditing. The same transparency that enables correction also creates attack surface. This tradeoff is inherent to all deterministic governance systems (see Limitations, Section 11).

A skill is a directory containing a `SKILL.md` file with YAML frontmatter and Markdown instructions:

```
skill-name/
+-- SKILL.md          # Required: frontmatter + instructions
+-- scripts/          # Optional: executable code
+-- references/       # Optional: additional documentation
+-- assets/           # Optional: templates, schemas
```

Frontmatter as Corrigibility Contract. The YAML frontmatter encodes the governance contract:

```
---
name: welfare-triage           # Required, 1-64 chars
description: Determines eligibility for welfare programs.
             Use when processing benefit applications.
license: Apache-2.0
compatibility: Requires access to citizen registry API
allowed-tools: Bash(curl:*) Read Write  # GOVERN constraint
metadata:
  author: state-welfare-dept
  version: "2.1"
---
```

The `allowed-tools` field is the **cybernetic anchor**: it specifies which tools the agent may invoke, hard-coded outside the LLM's context window. An LLM cannot prompt-inject past this constraint. It is enforced by the execution environment, not by the model's alignment.

Progressive Disclosure Architecture. Skills use progressive disclosure to manage context efficiently, directly implementing the LWD-R requirement:

6.4.1 Bounding Stochasticity

In learned systems, behavior is probabilistic. However, public infrastructure requires deterministic accountability. The Agentic Skills architecture resolves this tension by separating the *reasoning engine* (the stochastic model) from the *action boundary* (the deterministic skill definition).

Table 6: SKILL.md progressive disclosure layers

Layer	Tokens	When Loaded	LWD-R Mapping
Metadata	∼100	Startup (all skills)	L (Logic discovery)
Instructions	<5000	Activation	R (Representation)
Resources	As needed	Execution	D (Data provenance)

Principle 6.3 (Action Boundary Governance)**.** If the state cannot govern the neural network's weights, it must govern the action boundary. Agentic Skills provide the deterministic envelope that wraps around probabilistic inference.

When an agent attempts a state transition, the workflow orchestration does not rely on the model's internal alignment. Instead, it invokes a defined Skill that imposes exogenous constraints:

1. **Exception Handling (EXIT)**: The skill definition dictates explicit fallback paths and termination triggers, preventing the infinite loops that cause Governance Denial of Service (GDoS).
2. **Ontological Disclosure (CODE)**: By encoding the agent's workflow into declarative, human-readable files (e.g., YAML/Markdown skill definitions), the system's operational ontology becomes public. This satisfies the Representation (R) requirement of the LWD-R triad.
3. **Telemetry (AUDIT)**: Skills enforce structured inputs and outputs, transforming opaque "conversations" into discrete, auditable transaction logs.
4. **Binding Rules (GOVERN)**: The execution of the skill acts as the non-overrideable controller. An LLM cannot prompt-inject its way out of a hard-coded skill constraint.

6.4.2 Defeating Workflow Capture via Open Skills

The critical danger of agentic DPI is Epistemic Delegation: the loss of state sovereignty to proprietary orchestration layers. We previously defined the Workflow Capture Coefficient (WCC) as a function of reproduction cost, ontological substitutability, and portability.

Standardized Agentic Skills fundamentally disrupt this capture vector. If a state defines its administrative processes using open skill protocols rather than vendor-locked pipelines:

- $C_{\text{workflow}} \to 0$: The orchestration logic is transparent and easily replicable
- $O_{\text{substitutability}} \to 1$: Administrative categories are defined in open skill schemas, not hidden in a vendor's latent space
- $M_{\text{portability}} \to 1$: The state can swap the underlying LLM without losing or breaking the workflow logic

Consequently, adopting an open Agentic Skills architecture drives the WCC toward zero, preserving the state's interpretive sovereignty and satisfying the **FORK** test for the agentic era.

Claim 6.2. You don't govern the AI; you govern the Skills the AI is allowed to use. This is the architectural translation of corrigibility into the agentic domain.

Critical Limitation: Action Boundary Is Containment, Not Cure. The Action Boundary Protocol is a *necessary but not sufficient* condition for EPI corrigibility. It bounds the "act" but does not resolve deeper structural deficits:

1. **Inference-layer opacity persists.** The model's internal representation (the categories it uses to *interpret* inputs before selecting a skill) remains hidden. You govern what the

model *does*, not what it *thinks*. Representation Capture Risk (RCR) operates upstream of the Action Boundary.

2. **Compute capture blocks training forkability.** Skill portability enables swapping orchestration logic, but the underlying model remains compute-gated. If the LLM itself cannot be retrained by non-operators, the Action Boundary governs a black box.
3. **Temporal Variety Gap is unaddressed.** The model drifts from reality between retraining cycles. Bounding actions does not prevent the accumulating mismatch between frozen model variety and evolving environmental variety.
4. **Skill definitions encode ontological choices.** The categories embedded in skill schemas ("eligible," "fraudulent," "risk") are themselves epistemic commitments. Open skill protocols shift capture from the LLM to the skill author; they do not eliminate it.

The Action Boundary is therefore a **governance floor**, not a governance ceiling. It prevents the worst failure modes (GDoS, prompt injection, unbounded action) while leaving the deeper problem of epistemic sovereignty partially unresolved. Full EPI corrigibility requires the Action Boundary *plus* LWD-R disclosure *plus* training forkability: the complete stack, not any single layer.

Epistemic Constraint Boundary. Corrigibility preserves structural contestability but does not guarantee epistemic rigor, scientific discipline, or optimization efficiency. A system may satisfy corrigibility conditions and yet generate poorly designed experiments or unstable inferences. Corrigibility bounds structural error propagation; it does not ensure epistemic excellence.

6.4.3 Structural Gaps in Current Frameworks

Existing DPI-AI frameworks such as [Abadie, 2026] correctly identify composability and workflow orchestration as central challenges. However, they optimize for state capacity (efficient AI deployment) without addressing citizen capacity (structural correction rights). The corrigibility gaps are structural: no EXIT mechanism for refusing AI-mediated determinations; no LWD-R requirement for training transparency; no compute capture analysis; no citizen-corrective loop (GOVERN exists for operators, not subjects); and no FORK provision for training reproducibility.

Beyond corrigibility, the data governance provisions lack specificity on public domain status, licensing requirements, and sovereignty constraints. These gaps replicate the architectural failures documented in deterministic DPI.

7 Workflow Sovereignty and the Rule of Workflow

The final mutation of digital infrastructure is the transition from static registries to **Workflow Orchestration layers** (e.g., Palantir-style data fusion or agentic AI pipelines). In this regime, the operating layer of the state is no longer a ledger of records, but an orchestration of decisions.

7.1 From Rule of Law to Rule of Workflow

In classical governance, policy is defined in statute, executed by bureaucracy, and reviewed by courts. In a "Rule of Workflow" regime, policy becomes emergent from the orchestration logic: data schemas define reality, prompts define interpretation, and API contracts define valid inputs.

Workflow Capture is formalized as follows. Let W be the orchestration layer, M the underlying model, S the state policy function, and D the decision output. While classical governance assumes $S \to D$, AI-native governance operates as:

$$S \to W(M, \text{Data}) \to D \tag{5}$$

If W is proprietary and non-forkable, the state's ability to alter policy (ΔS) vanishes because outcomes are constrained by the workflow's technical affordances. Policy debate is reduced to parameter tuning within a vendor-controlled ontology.

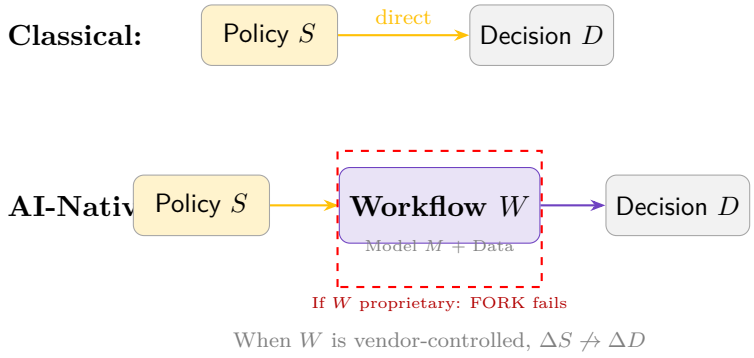

Figure 4: **Workflow Capture.** Classical governance: policy directly determines decisions. AI-native governance: policy is mediated by workflow orchestration. If the workflow is proprietary, policy changes cannot reliably change outcomes.

7.2 Measuring Epistemic Delegation

The **Workflow Capture Coefficient (WCC)** is a diagnostic abstraction illustrating weakest-dimension dominance across layers. It quantifies epistemic delegation risk as a function of reproduction cost, ontological substitutability, and portability. A WCC approaching 1 indicates near-total capture; a WCC approaching 0 indicates high sovereignty. The harmonic mean formulation ensures that weakness in any single dimension cannot be hidden by strength in others. Formal specification appears in Appendix A.

Claim 7.1. If a state procures a system with high WCC, it is not merely buying software; it is mathematically surrendering interpretive sovereignty to a vendor.

7.3 Fallout Governance

When AI decision throughput outruns human legislative correction velocity, the state enters **Fallout Governance**: policy ceases to guide behavior in advance; it instead reacts post-hoc to scandal, litigation, or failure clusters. Governance becomes damage containment: the Collingridge Dilemma at civilizational scale.

8 Architecture-Specific Risk Analysis

The preceding analysis enables architecture-specific risk diagnosis. Different model architectures exhibit distinct failure profiles across the five tests and the RCR dimension.

Interpretation.

- **Dense LLMs** fail FORK due to compute capture (κ). Training costs exceed accessible compute by orders of magnitude.
- **World models** present the highest RCR (lowest ΔO) because their representation schemas encode administrative ontologies that resist contestation.
- **Edge/SLM models** pass EXIT and FORK but may fail GOVERN due to fragmented deployment making coordinated governance infeasible.
- **Mixture of Experts** architectures may pass FORK if expert routing is documented, but gating opacity often violates CODE.

Table 7: **Topology-Aware Corrigibility Risk.** Impact of architectural class on structural tests. Notation: W = weights opacity, G = gating opacity, R = representation opacity, S = scale barrier, O = ontological capture, F = fragmentation, κ = compute capture.

Architecture	**EXIT**	**CODE**	ΔO **(RCR)**	**AUDIT**	**GOVERN**	**FORK**
Dense LLM	Partial	Fail (W)	Medium	Partial	Fail (S)	Fail (κ)
Mixture of Experts	Medium	Fail (G)	Medium	Fail (S)	Partial	**Pass**
World Model	Fail (O)	Fail (R)	**Very Low**	Partial	Partial	Fail (κ)
Edge / SLM	**Pass**	**Pass**	High	Fail (F)	Fail (F)	**Pass**

8.1 Edge Topologies: From Platform DPI to Protocol DPI

The framework addresses the transition from cloud-centric "Platform DPI" to decentralized "Protocol DPI" by analyzing how architectural choices like Edge computing and bytecode-based execution affect the five structural tests.

8.1.1 Edge AI: Servers vs. Mobiles

Edge AI is the primary strategy for restoring EXIT and FORK in population-scale infrastructure:

- **Server Edge (Federated Nodes)**: Running infrastructure across multiple independent server nodes (similar to Estonia's X-Road) ensures no single operator maintains execution monopoly. This creates Functional Exit Equivalence (FEE)[1]: if one node is captured, system variety is preserved through other federated nodes.
- **Mobile Edge (Local Inference)**: Transitioning to local execution on mobile devices (using Small Language Models) provides the strongest EXIT guarantee. Users perform essential functions without transmitting telemetry to central authority, bypassing Rule of Workflow capture.

Principle 8.1 (Edge Compute Capture Reduction)**.** Decentralization reduces Compute Capture risk: the cost of running infrastructure is distributed across the community's own hardware, lowering the resource invariance barrier κ.

8.1.2 CODE as Bytecode: Verification vs. Inspection

Bytecode and **Reproducible Builds** are the technical prerequisites for CODE in modern DPI:

- **Logic Verification**: Transparency is hollow if published source code does not match the binary running on edge devices. The framework requires Reproducible Builds so any third party can compile source into identical bytecode, verifying that "Logic" matches "Affidavit."
- **Wasm/Bytecode Portability**: Standardized, portable bytecode formats (like WebAssembly) avoid vendor lock-in. This lowers WCC because orchestration logic can be ported between cloud or edge environments, satisfying FORK.

Architectural Implication. By adopting Edge/Bytecode topologies, states move from **Rule of the Ledger** to **Rule of the Protocol**, ensuring that administrative power remains corrigible even as it scales toward universal necessity. The substrate shifts from centralized cloud to distributed edge, but the five tests remain invariant.

[1]FEE is defined and formalized in the companion paper (Section 6.7.2). When literal exit is impossible for essential services, FEE provides architectural guarantees that recreate the error-signal strength of market exit without requiring citizens to abandon society.

Table 8: Corrigibility impact of Edge/Bytecode topologies

Test	Edge/Bytecode Impact	Cybernetic Function
EXIT	**High** (local refusal possible)	Error Signal
CODE	**Verified** (via reproducible bytecode)	Transmission Clarity
AUDIT	Fragmented (requires federated telemetry)	Sensor
GOVERN	Challenging (requires federated guardrails)	Actuator
FORK	**Excellent** (low compute multiplier κ)	Selection Pressure

9 Anticipated Objections

Before discussing implications, we address two fundamental objections that challenge the framework's applicability to learned systems.

9.1 Objection: Deterministic Tests Cannot Apply to Stochastic Systems

A natural critique holds that applying rigid, deterministic structural tests to systems that are inherently stochastic constitutes a category error. How can tests designed for inspectable code apply to systems where identical inputs yield probabilistic outputs and "logic" is encoded in billions of weights?

Response: Verification Methods Evolve, Standards Remain. The framework acknowledges that learned systems dismantle the assumption that source code is the sole arbiter of behavior. However, this does not invalidate structural corrigibility. It *strengthens* its necessity. As systems become less intelligible, external correction mechanisms become more critical, not less.

The key insight is that the **standard remains invariant** while the **verification method adapts**:

- **CODE** shifts from reading source code to requiring the **LWD-R Triad**: Logic (architecture), Weights (parameters), Data (provenance), and Representation (ontological categories).
- **AUDIT** shifts from discrete logic inspection to statistical bounds and continuous monitoring.
- **FORK** shifts from file copying to resource forkability, measured by access to compute and training data, not just open weights.
- **GOVERN** requires deterministic guardrails *around* the stochastic model, governing objectives and value tradeoffs at the infrastructure layer.

Deterministic Boundaries as Engineering Requirement. The framework argues that deterministic structural boundaries are not incompatible with AI but are an *engineering requirement* for the agentic economy. Autonomous agents lack biological resilience for ambiguity. To an AI agent, the five tests function as a necessary API contract:

- EXIT = Exception handling (preventing logic deadlocks)
- CODE = Documentation (enabling valid plan formation)
- AUDIT = Observability (providing reliable failure signals)
- GOVERN = Constraint discovery (since "advisory" guidelines produce undefined behavior)

The Alternative is Surrender. If we accept that "AI is too stochastic to be evaluated by structural rules," we mathematically surrender to black-box monopolies. The framework does not attempt to inspect the probabilistic "thoughts" of AI models. Instead, it demands structural control over the *pipeline that builds them* (data, compute, representation) and the *infrastructure that deploys them* (workflow orchestrations and guardrails).

9.2 Objection: Compute Capture Makes FORK Trivially Impossible

If frontier model training costs exceed global academic compute capacity by orders of magnitude, doesn't the framework simply conclude "all frontier AI fails FORK," a trivially true and useless finding?

Response: The Distinction Is Training vs. Inference Forkability. The framework distinguishes *inference forkability* (running weights) from *training forkability* (reproducing the model). Only training forkability satisfies FORK for governance purposes. The evidence requirements in Table 5 are achievable: published training code, data manifests, and compute cost estimates.

The claim is not that all AI fails FORK, but that systems *withholding these artifacts* fail. OLMo, Pythia, and academic models demonstrate that FORK-compliant AI is possible. The question is whether frontier labs *choose* to comply.

The framework implies that **compute is itself DPI**. Legal forkability without practical compute accessibility is hollow. Public compute infrastructure becomes a governance requirement, not a reason to abandon the standard.

10 Epistemic Constraint Infrastructure

Corrigibility ensures that systemic error can be contested and corrected. It does not guarantee epistemic rigor within system operation. In verifiable domains such as software development, symbolic constraint systems (compilers, type systems, unit tests) provide hard pass/fail validation of outputs. These mechanisms enforce epistemic discipline through execution.

In learned or research-driven domains, comparable constraint infrastructure is often absent. Validation loss improvements, heuristic metrics, or informal review processes may not provide sufficient structural enforcement to prevent spurious inference.

Corrigible epistemic infrastructure therefore requires not only transparency and forkability, but engineered constraint mechanisms that reject inadequately controlled claims. Without such constraint, systems may remain structurally corrigible yet epistemically unstable.

10.1 Illustrative Example: Multi-Agent Research Organization

Consider a multi-agent research organization attempting to optimize a model parameter. The organization may satisfy structural corrigibility conditions: independent branches (FORK), transparent code (CODE), internal review (AUDIT), and supervisory governance (GOVERN). Yet if experimental protocols do not enforce baseline normalization, compute accounting, or ablation requirements, agents may report spurious improvements.

The system remains corrigible (errors can be contested), but epistemic progress is not guaranteed. This distinction clarifies the boundary between structural legitimacy and scientific competence.

Principle 10.1 (Corrigibility vs. Competence)**.** Structural corrigibility ensures that claims can be challenged and systems can be corrected. Epistemic quality requires additional constraint mechanisms that function as "compilers for reality," rejecting outputs that fail verification.

11 Discussion

11.1 Implications for AI Governance

This framework offers three contributions to AI governance discourse:

1. **Structural vs. Behavioral Focus.** Most AI governance frameworks focus on behavioral outcomes (bias, accuracy, harm). This framework focuses on structural preconditions: can affected parties correct the system regardless of what harms emerge? Structural corrigibility is necessary for any behavioral governance to function.

2. **Distinguishing Open-washing.** The LWD-R requirement and compute capture analysis provide falsifiable tests for "open AI" claims. Systems publishing weights without training data, code, or accessible compute achieve only inference forkability: performative transparency without correction capacity.

3. **Agentic Economy Readiness.** As autonomous agents proliferate, infrastructure must become machine-readable. The five tests, reinterpreted as API contracts, specify the minimum requirements for agentic interoperability. Incorrigible infrastructure blocks the agentic economy.

11.2 Limitations

1. **Threshold calibration is underdetermined.** The compute capture threshold κ, WCC trigger values, and RCR operationalization require empirical calibration that this paper does not provide.

2. **Rapid technological change.** AI architectures evolve faster than governance frameworks. The risk profiles in Table 7 may become obsolete as new architectures emerge.

3. **LWD-R is aspirational.** No current frontier model satisfies full LWD-R disclosure. The requirement describes what *should* exist for corrigibility, not what currently does.

4. **Compute accessibility is geopolitically constrained.** Public compute infrastructure sufficient for training forkability may be infeasible for many nations due to cost, expertise, and supply chain constraints.

5. **The framework does not address emergent capabilities.** Unpredictable capabilities that emerge during training present governance challenges that structural tests alone cannot address.

6. **Deterministic boundaries create known attack surfaces (SBOM problem).** The Agentic Skills architecture requires explicit, machine-readable disclosure of skill boundaries, allowed tools, and execution constraints. This transparency enables governance but simultaneously creates a complete map of the system's action space for adversaries. The same `allowed-tools` field that prevents unauthorized actions also reveals exactly which actions *are* authorized. This is analogous to Software Bill of Materials (SBOM) disclosure: useful for auditing dependencies but also useful for targeting known vulnerabilities. All deterministic governance systems face this tradeoff. The boundaries that constrain must be known, and what is known can be attacked. The framework does not resolve this tension; it makes it explicit.

11.3 Future Work

1. **Empirical RCR measurement**: Developing quantitative methods to assess ontological contestability
2. **Compute accessibility indices**: Cross-national comparison of training forkability capacity
3. **GDoS simulation**: Modeling governance failure under agentic scaling
4. **LWD-R schema standardization**: Machine-readable formats for disclosure requirements
5. **Regulatory integration**: Mapping framework tests to emerging AI regulations (EU AI Act, etc.)

12 Conclusion

The five corrigibility tests remain invariant for learned systems. What changes is the verification method and the resource barriers to compliance. This paper extends the framework through: LWD-R disclosure requirements extending CODE to training pipelines, Compute Capture as a resource barrier that transforms "open weights" into performative transparency, and the Action Boundary Protocol separating stochastic inference from deterministic execution.

Three implications:

1. **For policymakers**: The framework provides falsifiable tests for AI infrastructure procurement. Systems failing LWD-R or lacking training forkability should trigger governance scrutiny regardless of "open" marketing claims.
2. **For technologists**: The Agentic Skills standard offers a practical architecture for building corrigible AI systems. Reproducible skill definitions satisfy GOVERN without requiring interpretability of model internals.
3. **For citizens**: Infrastructure that affects populations at scale must be structurally correctable by those it governs, regardless of whether it executes deterministic code or learned parameters.

Liability and Enforcement. Architectural corrigibility must be complemented by clear liability and enforcement semantics: for any failure mode, supply-chain roles (principal, integrator, model provider, operator, trustee, as distinguished in EU AI Act Article 25 et seq.) must be mappable to legal responsibility and to mandated mitigation steps, so that evidence produces enforceable consequences rather than forensic narrative alone.

Adversarial Robustness. This framework assumes good-faith actors operating within institutional constraints. It does not address adversarial manipulation: coordinated gaming of audit mechanisms, governance capture through procedural compliance, prompt injection attacks on action boundaries, or fork-and-poison attacks on model provenance. Extension to adversarial settings requires additional machinery beyond this paper's scope.

Core Thesis. The corrigibility invariant extends to stochastic systems: systems remain correctable if and only if the five conditions close the feedback loop under probabilistic verification. Learned systems do not escape the requirement; they raise the verification burden. Corrigibility is not a deployment certification. It is a persistence condition.

A Agentic Skills Schema Specifications

To distinguish valid governance from performative compliance, the framework provides machine-readable assessment schemas for agentic workflows. These schemas convert the theoretical de-

fense against Workflow Capture into enforceable compliance gates.

A.1 Operator's Affidavit: `infrastructure.json`

The operator must declare exactly how the stochastic AI is bounded. The `agentic_workflow_governance` object proves they are using standard, deterministic skill definitions rather than proprietary black-box orchestration.

```
{
  "$schema": "https://anivar.net/corrigibility/epi/schema/infrastructure.json",
  "system_name": "State Welfare Triage Agent",
  "agentic_workflow_governance": {
    "workflow_type": "bounded_skill_orchestration",
    "skills_schema_url": "https://github.com/state-gov/welfare-skills",
    "protocol_standard": "agenticskills.io/v1",
    "deterministic_guardrails": {
      "pre_execution_validation": true,
      "post_execution_validation": true,
      "executed_outside_llm_context": true
    },
    "epistemic_delegation_metrics": {
      "ontology_substitutability": 0.9,
      "llm_agnostic": true,
      "vendor_lock_in_risk": "low"
    }
  }
}
```

Critical Fields.

- `protocol_standard`: References the open `SKILL.md` specification [Agent Skills, 2025], ensuring skills follow the standardized format adopted across 18+ platforms.
- `executed_outside_llm_context`: The cybernetic anchor. Legally binds the operator to a claim that governance constraints are hard-coded in the skill's execution environment (via `allowed-tools` in SKILL.md frontmatter), not merely injected into the LLM's system prompt (which can be bypassed).
- `llm_agnostic`: Asserts high model portability ($M_{\text{portability}}$), meaning the state can swap the underlying AI model without breaking the policy workflow.

A.2 Auditor's Finding: `corrigibility.json`

The auditor does not merely read the operator's JSON; they red-team it. The auditor attempts to bypass skill boundaries and calculates the actual Workflow Capture Coefficient (WCC).

```
{
  "$schema": "https://anivar.net/corrigibility/epi/schema/corrigibility.json",
  "target_system": "State Welfare Triage Agent",
  "workflow_capture_assessment": {
    "wcc_score": 0.15,
    "wcc_threshold_passed": true,
    "skill_boundary_tests": {
      "guardrails_bypassed_via_prompt_injection": false,
      "unauthorized_skill_invocation_possible": false,
```

```
      "termination_criteria_verifiable": true
    },
    "reproduction_tests": {
      "skills_reproducible_with_alt_llm": true,
      "ontology_schema_publicly_available": true
    }
  },
  "determinations": {
    "CODE": true,
    "GOVERN": true,
    "FORK": true
  }
}
```

A.3 Enforcement Logic Gates

The schema fields directly map to corrigibility test determinations:

1. **Enforcing GOVERN**: If `guardrails_bypassed_via_prompt_injection` is `true`, the system instantly fails GOVERN. The constraints were probabilistic (advisory to the LLM) rather than deterministic (binding).
2. **Enforcing CODE (LWD-R)**: If `ontology_schema_publicly_available` is `false`, the system fails the Representation (R) requirement of CODE, triggering Representation Capture Risk (RCR).
3. **Enforcing FORK**: If `skills_reproducible_with_alt_llm` is `false`, the workflow is tightly coupled to a proprietary vendor. The WCC score spikes, and the system fails FORK.

Table 9: Schema field to corrigibility test mapping

Schema Field	Test	Failure Condition
`executed_outside_llm_context`	GOVERN	`false` ⇒ constraints are advisory
`guardrails_bypassed_via_prompt_injection`	GOVERN	`true` ⇒ boundaries are soft
`ontology_schema_publicly_available`	CODE	`false` ⇒ RCR triggered
`skills_reproducible_with_alt_llm`	FORK	`false` ⇒ vendor lock-in
`wcc_score`	FORK	> 0.7 ⇒ high capture risk

Architectural Significance. By encoding Agentic Skills into adversarial JSON schemas, the framework transforms theoretical governance requirements into machine-readable, enforceable compliance gates. State architects can require these schemas in public procurement contracts, definitively banning black-box, vendor-locked agentic workflows.

Data and Code Availability

The complete framework, including schemas and assessment tooling, is available at:

- **Repository**: `https://github.com/anivar/corrigibility-framework`
- **Agentic Skills** (following the Agentic Skills specification [Agent Skills, 2025]):

  - `skills/schema-creator/`: For operators (infrastructure.json manifests) and auditors (corrigibility.json assessments). Supports both DPI and EPI with role-specific templates.
  - `skills/schema-evaluator/`: Validates published schemas, verifies score calculations (geometric mean, automatic F conditions), and detects contradictions between operator claims and auditor findings.
  - `skills/corrigibility-assessment/`: Rules and test definitions including LWD-R transparency criteria, Compute Capture detection, and Action Boundary verification.

- **DPI Schemas**: `schema/dpi/infrastructure.json`, `schema/dpi/corrigibility.json`
- **EPI Schemas**: `schema/epi/infrastructure.json`, `schema/epi/corrigibility.json`

The skill architecture demonstrates that the five tests remain invariant across DPI and EPI. Only the verification method and evidence requirements adapt to the system type. The separation into creator, evaluator, and assessment components enables independent tooling for operators, auditors, and validators.

Acknowledgments

This paper extends the DPI corrigibility framework [Companion Paper, 2026] to learned systems. The author thanks the AI governance research community for ongoing critique.

License

References

Ashby, W. Ross. *An Introduction to Cybernetics*. Chapman & Hall, 1956.

Bommasani, Rishi, et al. On the Opportunities and Risks of Foundation Models. *arXiv preprint arXiv:2108.07258*, 2021.

Gebru, Timnit, et al. Datasheets for Datasets. *Communications of the ACM*, 64(12):86–92, 2021.

Mitchell, Margaret, et al. Model Cards for Model Reporting. *Proceedings of FAT* 2019*, 220–229, 2019.

Soares, Nate and Fallenstein, Benja. Agent Foundations for Aligning Machine Intelligence with Human Interests. *Technical Report 2014–8*. Machine Intelligence Research Institute, 2017.

Hadfield-Menell, Dylan, et al. The Off-Switch Game. *Proceedings of the AAAI Workshop on AI, Ethics, and Society*, 2017.

Open Source Initiative. The Open Source AI Definition 1.0. `https://opensource.org/ai/open-source-ai-definition`, 2024.

Grother, Patrick, Ngan, Mei, and Hanaoka, Kayee. Face Recognition Vendor Test (FRVT) Part 3: Demographic Effects. *NIST Interagency Report 8280*, 2019.

Agent Skills Specification. The open SKILL.md standard for AI agent capabilities. `https://agentskills.io/specification`, 2025. Adopted by Claude, OpenAI Codex, GitHub Copilot, Cursor, and 18+ platforms.

European Parliament and Council. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence. *Official Journal of the European Union*, 2024.

Companion Paper. Corrigibility as a Structural Precondition for Digital Public Infrastructure: A Cybernetic Framework. *Working Paper*, 2026. Establishes the five-test framework (EXIT, CODE, AUDIT, GOVERN, FORK) for deterministic DPI.

Abadie, Daniel. DPI-AI Framework: Building AI-Ready Nations through Digital Public Infrastructure. Centre for Digital Public Infrastructure (CDPI), 2026. `https://digitalpublicinfrastructure.ai`